

PROJECT DEEP DIVE

Smart Maintenance Logbook

A railway inspection MVP that turns field transcripts, audio, and image evidence into structured defect logs and dashboard intelligence.

FastAPI backend

Next.js frontend

PostgreSQL / SQLite
ready

Whisper + Ollama optional

8101

Local backend

FastAPI API endpoint

3101

Local frontend

Next.js dashboard UI

3

Demo roles

Inspector, Supervisor, Manager

Manual inspection logs are hard to act on quickly

The MVP focuses on reducing reporting friction while preserving evidence and operational visibility.

Field reporting is messy

Inspectors may speak in Hindi, Kannada, English, or mixed phrases. The system needs to accept typed text, recorded audio, and uploaded audio.

Evidence is separated from the log

Images and audio clips are valuable, but they are usually stored away from the defect entry. This project keeps media linked to the defect record.

Supervisors need structured fields

A sentence like 'B2 brake pipe leakage' must become coach, component, defect type, severity, description, and status.

Managers need summary intelligence

The same defect stream should feed dashboards: open defects, critical defects, status distribution, component concentration, and recent risk.

The system is a working inspection workflow, not only a form

It has capture, extraction, evidence storage, review, status tracking, and analytics in one local demo.

FastAPI

Backend API

Routers, schemas, models, services

Next.js

Frontend UI

TypeScript dashboard experience

DB + files

Storage

Metadata in DB, media on disk

- **Inspector**
Creates a defect from transcript or audio and attaches image/audio evidence.
- **Supervisor**
Filters defects, reviews transcript and media, and updates status with remarks.
- **Manager**
Sees total/open/in-progress/resolved/critical counts and component/status analytics.
- **Intelligence layers**
Rule-based extraction is fast and deterministic; Whisper and Ollama are optional for speech and smarter extraction.

Three demo roles represent the railway maintenance workflow

The frontend role switcher changes the working perspective without adding complex authentication to the MVP.

Inspector: Ramesh Kumar

Logs defects from the field. Enters train, coach, location, transcript, audio, and image evidence.

Supervisor: Suresh Patil

Reviews incoming defects, checks media, filters by severity/status/component, and updates progress.

Manager: Anita Rao

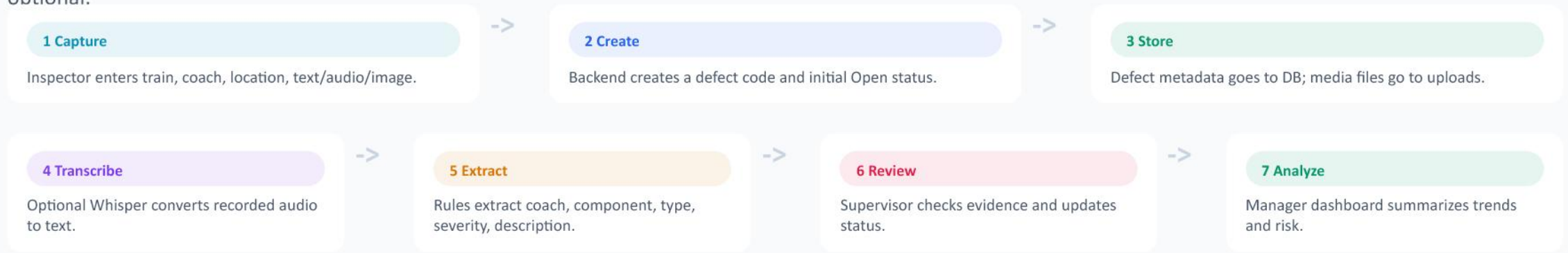
Uses analytics to understand workload, critical defects, component trends, and operational status.

From field observation to dashboard decision

Every screen and API exists to move one defect through capture, enrichment, review, and reporting.

A defect moves through seven clear stages

The design keeps the normal path fast and makes heavier intelligence features optional.



The project is layered so each responsibility is easy to explain

Frontend, API routers, services, database models, upload storage, Whisper, and Ollama are separated.

Browser UI

Next.js pages: Inspector, Supervisor, Manager. Uses Fetch API and browser MediaRecorder.

FastAPI app

Health, auth, defects, media, and dashboard routers. CORS allows the frontend origin.

Service layer

Storage, rule extractor, speech-to-text, and optional LLM extractor keep logic outside routers.

Database

SQLAlchemy models store users, defects, media metadata, and status history.

File storage

Audio and image files are stored under backend/uploads, with only paths stored in DB.

Optional intelligence

Whisper runs local speech-to-text. Ollama runs local LLM extraction only when requested.

The browser captures evidence; the backend processes and stores it

This split keeps the demo simple while leaving room for future on-device speech options.

Inspector device

- **Records audio**
MediaRecorder creates a browser audio file.
- **Previews image**
The UI shows selected image evidence before submit.
- **Sends requests**
Fetch API posts JSON and FormData to FastAPI.
- **Shows results**
The UI displays transcript, translation, extracted fields, and errors.

Backend server

- **Stores data**
SQLAlchemy writes defects, users, media rows, and status history.
- **Stores files**
Audio/image binaries are saved under backend/uploads.
- **Runs extraction**
Rules and optional Ollama convert transcripts into structured fields.
- **Runs Whisper**
When enabled, local backend Whisper converts audio to text/English.

FastAPI is the stable center of the MVP

It owns validation, persistence, media storage, extraction, transcription, and dashboard APIs.

The backend is organized around routers, models, schemas, and services

This makes it easy to point a senior to the exact place where each behavior lives.

app/main.py

Creates FastAPI, adds CORS, mounts uploads, includes routers, and exposes GET /health.

app/database.py

Creates SQLAlchemy engine/session, creates tables, and seeds demo users/defects.

app/models.py

Defines User, DefectLog, MediaFile, and StatusHistory database tables.

app/schemas.py

Defines Pydantic request/response shapes for API validation.

app/routers/*

Separates auth, defects, media upload, and dashboard endpoints.

app/services/*

Contains extractor, storage, speech, optional LLM extraction, and extraction pipeline logic.

The backend exposes focused API groups

Each group maps to one part of the operational workflow.

Group	Endpoints	Purpose
Health	GET /health	Shows API status, DB backend, ports, Whisper, and LLM flags.
Auth	GET /api/auth/demo-users	Returns seeded demo users for role switching.
Defects	POST/GET/PATCH /api/defects...	Creates, lists, retrieves, updates status, transcribes, and extracts defects.
Media	POST /media/audio, POST /media/image, GET /media	Uploads evidence and lists files linked to a defect.
Dashboard	/summary, /defects-by-component, /defects-by-status, /recent-defects	Feeds supervisor and manager summary views.

The database stores structured records; the file system stores binaries

This is deliberate because storing large audio/image blobs in the DB would make the MVP heavier.

User

id, name, employee_id, role, password_hash, created_at. Demo users support role switching.

DefectLog

defect_code, train, coach, component, type, severity, transcript, translation, status, location, timestamps.

MediaFile

defect_id, file_type, file_path, original_filename, uploaded_at. Links evidence to one defect.

StatusHistory

defect_id, old_status, new_status, remarks, updated_by, updated_at. Preserves supervisor workflow changes.

Data and evidence are stored in different places for the right reasons

Metadata needs querying; audio/images need durable file storage.

Database

- **Defect rows**
Train, coach, component, type, severity, status.
- **Transcript rows**
Raw transcript and translated English text.
- **Media metadata**
Original filename and relative file path.
- **Status history**
Every supervisor status change with remarks.

Uploads folder

- **Audio files**
Stored under backend/uploads/audio.
- **Image files**
Stored under backend/uploads/images.
- **Safe filenames**
UUID prefix and extension validation.
- **Static serving**
FastAPI mounts /uploads so UI can play or preview evidence.

Fast rules first, optional AI second

The project avoids blocking the MVP on heavy models while still allowing smarter refinement.

The extractor converts known railway phrases into structured fields

It does not guess from audio. It receives text and applies transparent rules.

Coach detection

Regex detects S1/S2/S3, B1/B2, A1/A2, D1, C1 and normalizes values like S-3 to S3.

Component rules

Door, brake, wheel, electrical, seat, toilet/water, window, and leakage keywords map to component names.

Severity rules

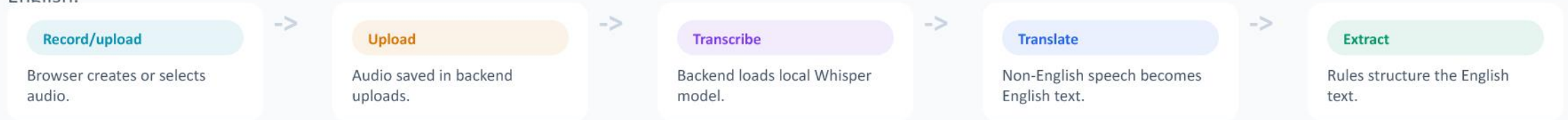
Brake leakage, wheel crack, smoke, fire, and electrical spark become Critical; damaged/not closing becomes Medium.

Defect type rules

The extractor returns short labels like Door not closing, Brake leakage, Electrical spark, or Seat damage.

Whisper is optional, local backend speech-to-text

When enabled, it converts audio into text and can translate Hindi/Kannada audio into English.



Important: Whisper is not a cloud service in this MVP. It runs on the backend machine using local model files. If `ENABLE_WHISPER=false` or Whisper is not installed, the app still works with typed transcripts.

Hindi and Kannada audio are translated into English before extraction

The extractor rules are English-focused, so translation gives it a consistent input language.

Auto detect

Useful for quick demos, but less reliable when audio is noisy or language is mixed.

Select Hindi/Kannada

The UI lets the user choose language so the backend can use the stronger non-English Whisper model.

English output

The backend saves `raw_transcript` and `translated_text`, then extracts from the best English text available.

The LLM is used as a refinement layer, not the default critical path

It can interpret more flexible wording, but the app keeps rule extraction as a safe fallback.

Local model

Ollama runs on localhost with llama3.2:latest. No cloud LLM call is required in this setup.

Structured JSON

The backend asks the LLM to return only coach, component, type, severity, and description.

Validation

Pydantic validates the LLM output and rejects unsupported component/severity values.

Fallback

If Ollama is slow, unavailable, or invalid, rule-based extraction remains the reliable result.

A field workflow should not wait for heavy AI when rules can give a good first result

The project separates immediate logging from optional intelligence refinement.

Default path

- **Typed transcript**
Immediate defect creation.
- **Rule extraction**
Fast coach/component/severity result.
- **Media upload**
Audio/image evidence attached.
- **Dashboard update**
Supervisor and manager can see it quickly.

Optional AI path

- **Whisper**
Transcribes and translates audio when enabled.
- **Ollama**
Improves extraction only when user requests it.
- **Confidence guard**
Weak transcription avoids overwriting defect fields.
- **Fallback**
Rules protect the demo if local AI is slow.

The UI is role-based and demo-ready

Each page is designed around what that user needs to do next.

Next.js pages are supported by reusable dashboard components

The API client centralizes backend calls so pages stay focused on user experience.

app/inspector/page.tsx

Inspector workspace for creating defects from transcript/audio/image evidence.

app/supervisor/page.tsx

Supervisor workspace for filtering, reviewing media, transcribing evidence, and updating status.

app/manager/page.tsx

Manager analytics view with summary cards, component buckets, status buckets, and critical defects.

components/*

AppShell, Sidebar, DefectForm, DefectTable, AudioRecorder, ImageUpload, StatCard, LoadingBlock, EmptyState.

lib/api.ts

Single API client for health, demo users, defects, media, transcription, extraction, and dashboard endpoints.

lib/types.ts

Frontend TypeScript types mirror backend response payloads for safer UI development.

The Inspector page is optimized for fast defect capture

It supports typed transcript, audio recording, audio upload fallback, image preview, and extracted result display.



If audio is submitted and Whisper is enabled, the UI first saves the defect, uploads audio, transcribes it, shows the transcript, then runs fast extraction. AI refinement is a separate button.

The Supervisor page turns captured logs into action

It is built for reviewing evidence, filtering defects, and updating progress.

Summary cards

Total, open, in-progress, resolved, and critical counts give instant workload status.

Defect table

Filter by status, severity, and component, then select a defect for details.

Evidence review

Supervisor sees transcript, English translation, audio playback, and image previews.

Status update

Status dropdown plus remarks writes to DefectLog and StatusHistory.

The Manager page aggregates the defect stream into decisions

It focuses on trends and risk rather than individual form entry.

All

Total defects

Complete captured log volume

Risk

Critical defects

Brake, wheel, smoke, fire, electrical spark

Closure

Resolved defects

Work completed and tracked

- **Defects by component**
Shows concentration areas like Brake System, Door, Electrical System, Seat/Berth.
- **Defects by status**
Shows whether issues are stuck Open, In Progress, or moving to Resolved.
- **Recent critical defects**
Gives leadership immediate visibility into safety-sensitive observations.
- **Future extension**
The same model can support date filters, depot filters, SLA charts, and export reports.

Why these methods were chosen

The project balances demo speed, local privacy, reliability, and future production paths.

Each tool was selected for a practical MVP reason

The choices keep the system understandable and replaceable.

FastAPI

Fast to build, typed request/response validation with Pydantic, good for Python AI/service integration.

SQLAlchemy

Clean database models and sessions, with ability to switch SQLite/PostgreSQL through DATABASE_URL.

Next.js + TypeScript

Professional dashboard UI, component reuse, type safety, and clean API integration.

Local uploads

Simple MVP evidence storage without adding Docker, S3, or infrastructure too early.

Whisper optional

Adds speech-to-text while keeping typed transcript workflow available on low-resource machines.

Ollama optional

Adds local LLM intelligence without making cloud LLM dependency mandatory.

There are multiple ways to build this; the MVP chose the lowest-friction path

These alternatives become useful when moving from demo to production.

On-device speech

Possible in a mobile app using native SDKs or small models. Reduces upload latency but increases device requirements.

Cloud speech API

Usually faster and more accurate at scale, but needs internet, vendor cost, and data/privacy review.

Cloud LLM extraction

Could be more accurate and faster than local CPU Ollama, but introduces external dependency and cost.

Background jobs

Better production UX: submit immediately, process audio/AI in queue, notify UI when complete.

Object storage

S3/Azure Blob/Internal NAS is better than local uploads for production evidence retention.

Full auth/RBAC

Needed for production so each user only performs allowed actions and every change is auditable.

The slow parts are model loading and multilingual speech processing

The project protects the UX by keeping the first structured result fast.

Fast operations

Typed transcript extraction, defect creation, media metadata saving, listing defects, dashboard summaries, and status updates.

Slower operations

Whisper model loading, Hindi/Kannada transcription, translation, and local Ollama extraction on CPU.

Mitigation

Submit fast first, show transcript early, run extraction separately, make AI refinement optional, and keep rule fallback.

A clean live demo should start with the reliable path

Show the complete workflow first, then show optional voice and AI features.

- **1. Start backend**
Run FastAPI on 127.0.0.1:8101 and confirm /health is ok.
- **2. Start frontend**
Run Next.js on 127.0.0.1:3101 and open the Inspector page.
- **3. Submit typed transcript**
Use 'Coach B2 brake pipe leakage...' to show Critical extraction quickly.
- **4. Review as supervisor**
Open Supervisor, select the new defect, inspect transcript/media, update status.
- **5. Review as manager**
Open Manager to show changed summary and component/status analytics.
- **6. Optional voice/AI**
Record audio, transcribe to text, then optionally click Improve with AI.

The MVP is complete enough to validate, but production needs hardening

These are the next upgrades before real deployment.

Authentication and RBAC

Replace demo role switcher with real login, permissions, audit identity, password policy, and sessions.

Background processing

Queue audio transcription and AI extraction so the UI remains responsive for large files.

Model strategy

Choose local GPU Whisper/Ollama, cloud speech/LLM, or hybrid based on cost, privacy, accuracy, and latency.

Deployment architecture

Use managed PostgreSQL, backend service, frontend hosting, HTTPS, domain, and proper environment management.

Durable media storage

Move uploads to object storage or enterprise file storage with retention and access controls.

Operational features

Assignments, notifications, SLA, search, export, audit logs, mobile experience, and monitoring.

This MVP proves the inspection-to-decision loop

The strongest story is not only AI; it is reliable structured maintenance workflow with optional intelligence.

What to say

We built a working railway maintenance logbook where inspectors can capture defects with transcript, audio, and image evidence; the backend structures the report; supervisors act on it; managers see analytics.

- **Reliability**
Rule-based extraction keeps the core demo fast and explainable.
- **Intelligence**
Whisper and Ollama are integrated as optional local AI layers.
- **Traceability**
Defect, evidence, user, and status history are stored as separate records.
- **Scalability path**
The same architecture can move to real auth, background jobs, object storage, and production DB.